ASSESSMENT TOOL 1

CO3 AT1 EXPERIMENTS

NAME: SHANI PRIYA

REG NO: 192371033

COURSE CODE: CSA5110

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

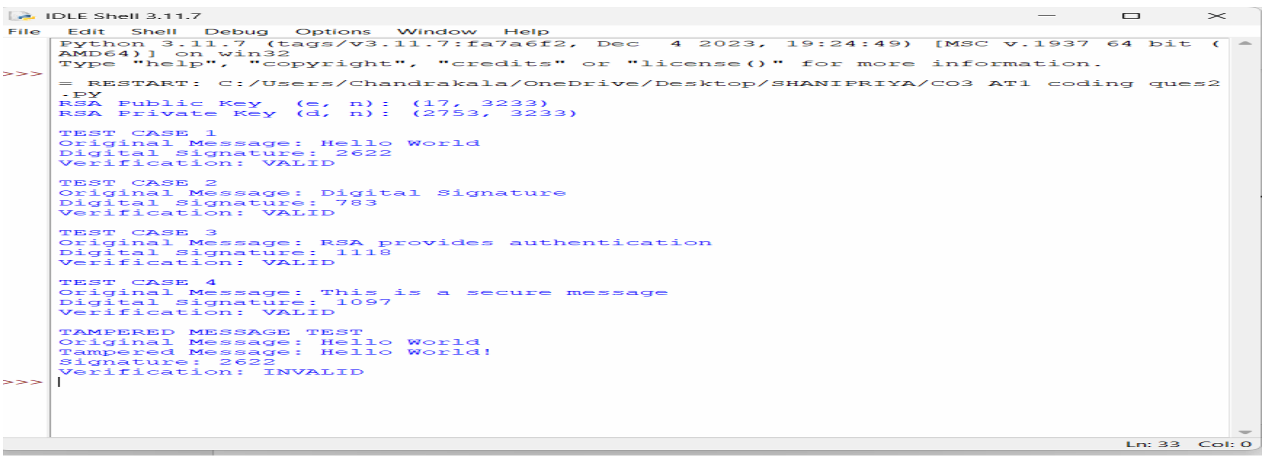
1. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

CODE:

```
import hashlib
import math
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = pow(e, -1, phi)
print("RSA Public Key  (e, n):", (e, n))
print("RSA Private Key (d, n):", (d, n))
def sign_message(message):
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_integer = int(hash_value, 16)
    signature = pow(hash_integer, d, n)
    return signature
def verify_signature(message, signature):
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_integer = int(hash_value, 16)
    decrypted_signature = pow(signature, e, n)
    return decrypted_signature == (hash_integer % n)
test_messages = [
    "Hello World",
    "Digital Signature",
```

```
    "RSA provides authentication",
    "This is a secure message"
]
for i, message in enumerate(test_messages, 1):
    print("\nTEST CASE", i)
    print("Original Message:", message)
    signature = sign_message(message)
    print("Digital Signature:", signature)
    if verify_signature(message, signature):
        print("Verification: VALID")
    else:
        print("Verification: INVALID")
print("\nTAMPERED MESSAGE TEST")
message = "Hello World"
signature = sign_message(message)
tampered_message = "Hello World!"
print("Original Message:", message)
print("Tampered Message:", tampered_message)
print("Signature:", signature)
if verify_signature(tampered_message, signature):
    print("Verification: VALID")
else:
    print("Verification: INVALID")
```

OUTPUT:



```
IDLE Shell 3.11.7
File Edit Shell Debug Options Window Help
Python 3.11.7 (tags/v3.11.7:fa7a6f2, Dec  4 2023, 19:24:49) [MSC v.1937 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Chandrakala/OneDrive/Desktop/SHANIPRIYA/CO3 AT1 coding ques2
.py
RSA Public Key  (e, n): (17, 3233)
RSA Private Key (d, n): (2753, 3233)

TEST CASE 1
Original Message: Hello World
Digital Signature: 2622
Verification: VALID

TEST CASE 2
Original Message: Digital Signature
Digital Signature: 783
Verification: VALID

TEST CASE 3
Original Message: RSA provides authentication
Digital Signature: 1118
Verification: VALID

TEST CASE 4
Original Message: This is a secure message
Digital Signature: 1097
Verification: VALID

TAMPERED MESSAGE TEST
Original Message: Hello World
Tampered Message: Hello World!
Signature: 2622
Verification: INVALID
>>>
Ln: 33 Col: 0
```

2. Write a Python program to generate the MD5 hash value of a given text message.

Input: “HELLO WORLD”

Output: 32-character hexadecimal MD5 digest.

CODE:

```
import hashlib
messages = [
    "Hello World",
    "Hello",
    "Python",
    "Digital Signature",
    "123456789"
]
for message in messages:
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    print("Original Message:", message)
    print("MD5 Hash:", md5_hash)
    print("Length:", len(md5_hash), "characters")
    print("-" * 50)
```

OUTPUT: